2.3. Identificación de los ámbitos de utilización de las variables (Criterio 2.c)

- **Resultado de Aprendizaje 2 (RA2):** *Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores Web.*

- **Criterio Curricular Oficial: CE 2.c** (*Se han identificado los ámbitos de utilización de las variables*).

- **Ponderación del Criterio:** (**20% del RA2** | **1% sobre la calificación final del módulo**).

A. ¿Qué es el Ámbito (*Scope*) de una Variable?

El **ámbito** (*scope*) es el conjunto de reglas que determinan en qué partes del código una variable es accesible y visible, y en qué momento es destruida por el recolector de basura (*Garbage Collector*) de la memoria RAM.

En JavaScript existen tres niveles fundamentales de ámbito:

1. **Ámbito Global (*Global Scope*):** Accesible desde cualquier lugar del script.
2. **Ámbito de Función (*Function / Local Scope*):** Accesible únicamente dentro de la función donde se declaró.
3. **Ámbito de Bloque (*Block Scope*):** Accesible únicamente dentro del par de llaves { ... } donde se definió.

B. Los Tres Niveles de Ámbito

1. Ámbito Global

Cualquier variable declarada fuera de una función y fuera de un bloque de llaves pertenece al ámbito global. Todos los scripts de la página pueden leerla y modificarla:

```
let nombreAplicacion = "Portal Educativo IES"; // Variable Global

function mostrarCabecera() {
  console.log("Aplicación:", nombreAplicacion); // Lectura
permitida desde dentro de la función
}

mostrarCabecera();
console.log("En el programa principal:", nombreAplicacion); //
Lectura permitida en el exterior
```

Ciclo de vida de las variables globales: Nacen en el momento en que se procesa o carga el script en el navegador y permanecen en memoria durante toda la sesión de la página, destruyéndose únicamente cuando el usuario cierra la pestaña o navega hacia otra URL.

El riesgo de la polución global: Si dos scripts independientes usan la misma variable global (por ejemplo, `var total = 0;`), uno sobrescribirá el valor del otro de forma silenciosa, originando fallos catastróficos.

Diferencia entre var y let/const en el Ámbito Global:

- En el navegador, una variable declarada con var en el ámbito global se convierte automáticamente en una propiedad del objeto global window (window.miVariable).
- Si se declara con let o const, sigue siendo accesible globalmente pero **no contamina el objeto window**, evitando sobreescribir métodos nativos del navegador como window.open o window.close.

2. Ámbito de Función (Ámbito Local Clásico)

Cuando declaras una variable dentro del cuerpo de una función, dicha variable nace al invocarse la función y se destruye al terminar su ejecución. Este es el único ámbito que respetaba la palabra reservada clásica var:

```
function autenticarUsuario() {
  var tokenSesion = "XYZ-123456"; // Ámbito de función (local)
  console.log("Token generado dentro:", tokenSesion); // Funciona
}

autenticarUsuario();

// Intentar acceder desde fuera de la función:
// console.log(tokenSesion);
// ERROR: Uncaught ReferenceError: tokenSesion is not defined
```

3. Ámbito de Bloque (ES6+: let y const)

Un bloque de código es cualquier conjunto de sentencias delimitadas por llaves { ... }, como el cuerpo de un if, un switch, un bucle for/while o simplemente llaves aisladas.

- **let y const tienen ámbito de bloque:** Solo son visibles entre las llaves donde se declararon.
- **var IGNORA el ámbito de bloque:** Se escapa de las llaves y se filtra al exterior (fuga de variables).

```
// A. Demostración con bloque simple
{
  var variableConVar = "Me escapo de las llaves"; //
  let variableConLet = "Quedo atrapada dentro"; //
  const variableConConst = "También atrapada dentro"; //
```

```
}

console.log(variableConVar);   // Funciona: "Me escapo de las
llaves"
// console.log(variableConLet);   // ERROR: Uncaught
ReferenceError: variableConLet is not defined
// console.log(variableConConst); // ERROR: Uncaught
ReferenceError: variableConConst is not defined

// B. El clásico problema del bucle contador
for (var i = 0; i < 3; i++) { //
  // Bucle con var
}
console.log("Valor de i tras el bucle (var):", i); // Imprime: 3
(¡La variable se filtró al exterior!)

for (let j = 0; j < 3; j++) {
  // Bucle con let
}
// console.log(j); // ERROR: ReferenceError: j is not defined (i
quedó destruida limpiamente)
```

C. Sombreado de Variables (*Variable Shadowing*) y Cadena de Ámbitos (*Scope Chain*)

Cuando una función o bloque necesita resolver el valor de una variable, el motor busca hacia afuera siguiendo una jerarquía estricta denominada **Cadena de Ámbitos** (*Scope Chain*):

1. Busca primero en el ámbito local más inmediato (el bloque o función actual).
2. Si no la encuentra, sube al ámbito exterior envolvente.
3. Repite el proceso hasta alcanzar el Ámbito Global.
4. Si llega al ámbito global y no existe, lanza un ReferenceError.

Si una variable local tiene exactamente el mismo nombre que una variable global, la variable local **ensombrece** (*shadowing*) a la global mientras se esté dentro de su ámbito, sin modificar el valor de la exterior:

```
let curso = "2° DAW (Ámbito Global)"; // Variable Global

function gestionarAula() {
  let curso = "1° SMR (Ámbito Local de la función)"; // Ensombrece
a la variable global

  if (true) {
    let curso = "Taller de Robótica (Ámbito de Bloque)"; //
Ensombrece a la de la función
    console.log("Nivel 3 (Bloque):", curso); // Imprime: Taller de
Robótica
  }
```

```
  console.log("Nivel 2 (Función):", curso);  // Imprime: 1º SMR
(vuelve al valor de la función)
}

gestionarAula();
console.log("Nivel 1 (Global):", curso);      // Imprime: 2º DAW
(la global permaneció intacta)
```

D. Elevación (*Hoisting*) y la Zona Muerta Temporal (*TDZ*)

El comportamiento de cómo se cargan las variables en memoria antes de ejecutarse el script marca una diferencia crítica entre var, let y const:

1. Elevación con var

El motor detecta las declaraciones con var y reserva espacio para ellas al inicio del contexto, asignándoles el valor provisional undefined:

```
console.log(precioArticulo); // Imprime: undefined (NO da error;
fue elevada sin valor)
var precioArticulo = 50;
console.log(precioArticulo); // Imprime: 50
```

2. Elevación con let y const: La Zona Muerta Temporal (*TDZ*)

let y const también se elevan internamente, pero el motor prohíbe terminantemente acceder a ellas antes de la línea física de su inicialización. El tramo de código comprendido entre el inicio del bloque y la línea de declaración se denomina **Zona Muerta Temporal (*Temporal Dead Zone - TDZ*)**:

```
{
  // INICIO DE LA TDZ PARA 'tasa'
  // console.log(tasa); // ERROR: Uncaught ReferenceError: Cannot
access 'tasa' before initialization

  let datoNeutro = 10;
  // FIN DE LA TDZ PARA 'tasa'
  let tasa = 0.21; // Aquí nace formalmente la variable
  console.log(tasa); // Imprime: 0.21
}
```

E. Fugas de Ámbito (*Scope Leaks*) y el Modo Estricto ("use strict")

En JavaScript clásico, si un programador olvida escribir let, const o var delante de una variable, el motor no detiene la ejecución; en su lugar, **crea una variable global oculta en window**, originando fugas de memoria y errores impredecibles:

```
function procesarCalculo() {
  // ERROR TIPOGRÁFICO: Se omite var/let/const
```

```
  resultadoFuga = 100; // Se convierte automáticamente en variable
global accidental
}

procesarCalculo();
console.log(window.resultadoFuga); // Imprime: 100 (La variable
contaminó el entorno global)
```

Prevención Profesional: "use strict"

Al colocar la directiva "use strict"; en la primera línea de un archivo o función, el motor activa el **modo estricto** y bloquea de inmediato cualquier variable no declarada:

```
"use strict"; // Activa las restricciones de seguridad modernas

function procesarCalculoSeguro() {
  // total = 200; // ERROR: Uncaught ReferenceError: total is not
defined
  let total = 200; // Obliga a declarar el ámbito correctamente
  console.log(total);
}
procesarCalculoSeguro();
```

F. El Patrón Módulo mediante IIFE (*Immediately Invoked Function Expressions*)

Antes de la llegada de los módulos nativos en ECMAScript 2015 (import / export), la única herramienta que ofrecía JavaScript para aislar variables del objeto global window era el **ámbito de función**. Si un desarrollador incluía múltiples archivos .js en un mismo documento HTML, cualquier variable declarada con var fuera de una función pasaba a formar parte del espacio de nombres global (window), provocando colisiones accidentales de identificadores.

Para solucionar este problema se diseñó el patrón **IIFE** (Expresión de Función Invocada Inmediatamente).

Estructura Sintáctica de una IIFE

Una IIFE es una función que se declara y se ejecuta en el mismo instante en que el intérprete del navegador la lee. Para que el motor no la interprete como una declaración de función tradicional (la cual requeriría un nombre obligatorio), se encierra entre paréntesis (...) para transformarla en una **expresión de función**, seguida de otro par de paréntesis () para ejecutarla:

```
(function() {
  // Código completamente aislado
  var secreto = "ClavePrivada123";
  console.log("IIFE ejecutada con éxito");
})();
```

```
// Si intentamos acceder desde fuera:
// console.log(secreto); // Lanza ReferenceError: secreto is not
defined
```

G. ACTIVIDADES

Ejercicio 1: El Dilema del Bucle Asíncrono con `var` frente a `let`

Enunciado: Analiza el siguiente fragmento de código con un temporizador `setTimeout` y predice qué imprimirá en la consola. ¿Has acertado?.

```
for (var i = 0; i < 3; i++) {
  setTimeout(function() {
    console.log("Valor con var:", i);
  }, 100);
}

for (let j = 0; j < 3; j++) {
  setTimeout(function() {
    console.log("Valor con let:", j);
  }, 100);

}
```

Ejercicio 2: Identificación de la Zona Muerta Temporal (TDZ)

Enunciado: Analiza el siguiente fragmento de código y predice qué imprimirá en la consola. ¿Has acertado?.

```
let usuario = "Carlos";

function configurarPerfil() {

  console.log("El usuario actual es:", usuario);

  let usuario = "Marta";

}

configurarPerfil();
```

Ejercicio 3. Actividad Práctica Guiada de Laboratorio: Simulador Visual de Ámbitos y Prevención de Fugas de Memoria

Se entrega una interfaz interactiva donde el alumnado introduce nombres y valores de variables para comprobar en vivo qué elementos sobreviven fuera de los bloques y funciones, detectando fugas accidentales al objeto global.

Código Base: `laboratorio_ambitos.html`

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Laboratorio CE 2.c: Ámbitos de Variables</title>
  <style>
    body { font-family: monospace; margin: 30px; background-color: #0f172a; color: #f8fafc; }
    .panel { max-width: 650px; background: #1e293b; padding: 25px; border-radius: 8px;
border: 1px solid #334155; }
    h2 { color: #38bdf8; margin-top: 0; }
    button { padding: 10px 16px; background-color: #0284c7; color: white; border: none;
border-radius: 4px; cursor: pointer; font-weight: bold; }
    button:hover { background-color: #0369a1; }
    .terminal { margin-top: 20px; padding: 15px; background: #020617; border-radius: 4px;
border-left: 4px solid #38bdf8; font-size: 13px; line-height: 1.6; white-space: pre-wrap; }
    .resaltado-ok { color: #4ade80; }
    .resaltado-error { color: #f87171; }
  </style>
</head>
<body>

  <div class="panel">
    <h2>Laboratorio CE 2.c: Comprobador de Ámbitos</h2>
    <p>Ejecuta el script de prueba para rastrear la visibilidad de variables locales y de
bloque.</p>
    <button id="btnEjecutar">Ejecutar Diagnóstico de Ámbitos</button>
    <div id="pantallaTerminal" class="terminal">Esperando ejecución...</div>
  </div>

  <script src="laboratorio_ambitos.js"></script>
</body>
</html>
```

Código Base: `laboratorio_ambitos.js`

```
const visor = document.getElementById("pantallaTerminal");

function registrar(texto, clase = "") {
  const linea = document.createElement("div");
  if (clase) linea.className = clase;
  linea.textContent = texto;
  visor.appendChild(linea);
}
```

```
document.getElementById("btnEjecutar").addEventListener("click", () => {
  visor.innerHTML = "";
  registrar("=== INICIANDO AUDITORIA DE AMBITOS ===");

  // 1. Ámbito de Bloque Aislado
  {
    var testigoVar = "Creada con var en bloque";
    let testigoLet = "Creada con let en bloque";
  }

  // Comprobación de visibilidad de testigoVar
  if (typeof testigoVar !== "undefined") {
    registrar("✓ 'testigoVar' se fugó del bloque: " + testigoVar, "resaltado-ok");
  } else {
    registrar("✗ 'testigoVar' no es accesible.", "resaltado-error");
  }

  // Comprobación de visibilidad de testigoLet
  try {
    let lectura = testigoLet;
    registrar("✓ 'testigoLet' es accesible: " + lectura, "resaltado-ok");
  } catch (e) {
    registrar("✗ 'testigoLet' fue destruida al salir del bloque (ReferenceError).",
"resaltado-error");
  }

  // 2. Comprobación de Contaminación del Objeto Window
  var variableGlobalVar = "Dato con var";
  let variableGlobalLet = "Dato con let";

  if (window.variableGlobalVar === "Dato con var") {
    registrar("⚠ 'variableGlobalVar' contaminó el objeto global window.", "resaltado-error");
  }

  if (typeof window.variableGlobalLet === "undefined") {
    registrar("✓ 'variableGlobalLet' NO contamina window (Entorno limpio).", "resaltado-ok");
  }

  registrar("=== FIN DE LA AUDITORIA ===");
});
```

Modificaciones Solicitadas al Alumnado:

1. **Modificación 1 (Simulación de Sombreado - Shadowing):** Modifica el script para declarar una variable global `const TASA = 0.21;` y dentro de una función calcular una tarifa temporal sombreando la constante con `const TASA = 0.10;`, imprimiendo ambos valores para comprobar que la tasa original no se altera.

2. **Modificación 2 (Activación de "use strict"):** Modifica el inicio del script para incorporar la directiva `"use strict";` e intenta asignar un valor a una variable sin declararla (`saldoSinDeclarar = 500;`). Captura el error con un bloque `try/catch` y muéstralo en la pantalla del laboratorio con la clase `.resaltado-error`.